Avionics RTOS Task Architecture: Inter-Task Communication and Priority Inheritance

Jose

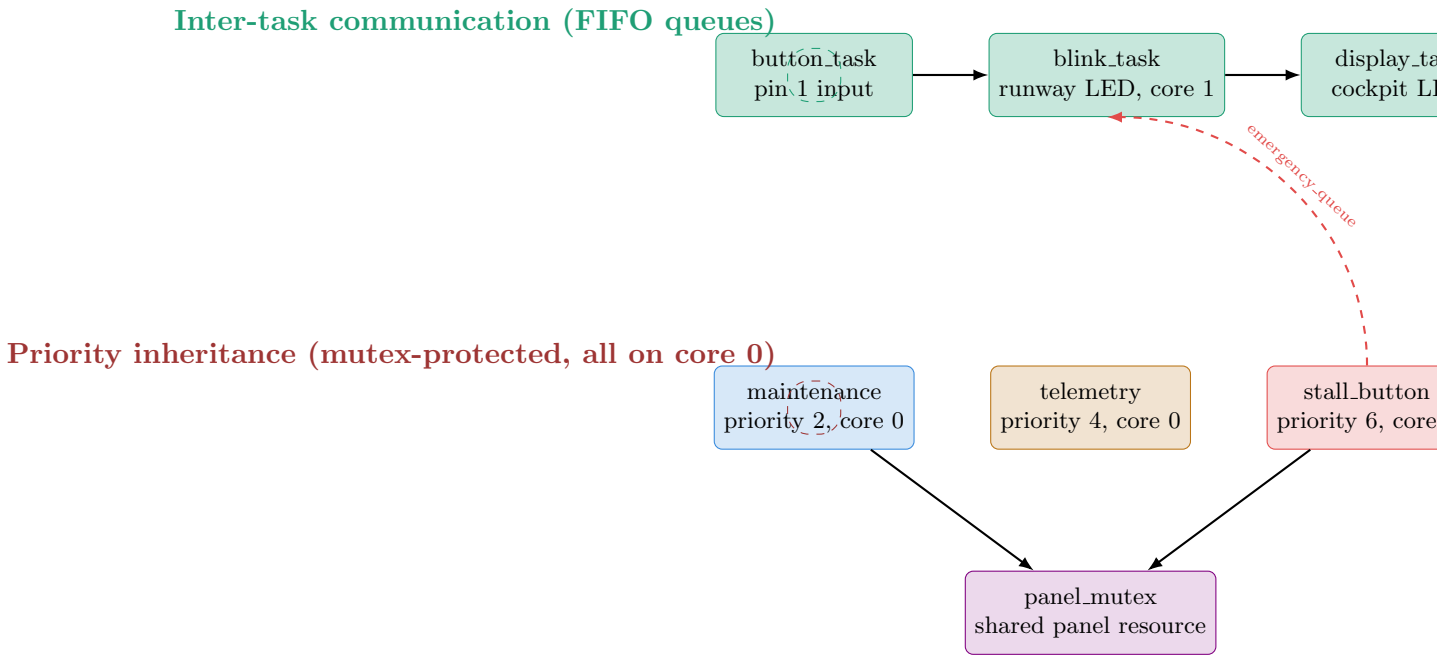
July 31, 2026

1 System Architecture

The system is organized into two task clusters running on the dual-core ESP32-S3 under FreeRTOS: an inter-task communication (ITC) cluster using FIFO queues, and a priority-inheritance cluster using a mutex-protected shared resource. The clusters are largely independent, with one deliberate exception: a dedicated `emergency_queue` allows `stall_button_task` to override `blink_task`'s runway lighting behavior directly, bypassing the routine pilot-request path entirely so the emergency response can never be delayed by queued pilot traffic.

Core assignment

`blink_task` is pinned to core 1 (APP_CPU); `button_task` and `display_task` are left unpinned. The entire priority-inheritance cluster — `maintenance_task`, `telemetry_task`, and `stall_button_task` — is deliberately pinned to core 0 (PRO_CPU) *together*. This is required, not incidental: priority ordering, and the boost that priority inheritance grants, only governs contention between tasks competing for the same core's execution timeline. Left unpinned, the scheduler can place these tasks on different cores, where they run genuinely concurrently regardless of priority, making any comparison between the mutex and binary-semaphore configurations meaningless — two tasks on separate cores are never actually contending for anything. Pinning all three to one core was necessary before the inheritance-on/off comparison in Section 5 could produce valid evidence.



Notes: `telemetry_task` has no arrow into `panel_mutex` — it never contends for the resource directly, but because it shares core 0 with `maintenance_task` and `stall_button_task`, it can still delay the mutex holder by consuming CPU time on that shared core when priority inheritance is not active (Section 5). The dashed red path is the emergency override: a separate queue from the routine pilot-request path, so an emergency signal can never be blocked behind queued pilot traffic. Note that the cockpit panel confirmation (`panel_mutex` access) is *not* bypassed by this path — only the physical runway lighting response is unconditional; the secondary panel/log confirmation remains subject to the bounded wait guaranteed by priority inheritance.

2 Task Table with WCET Evidence

Worst-case execution time (WCET) for each task was measured directly on hardware using `esp_timer_get_time()` (microsecond-resolution) wrapped around each task's actual computation, explicitly excluding any `vTaskDelay`/`xSemaphoreTake` blocking calls (which yield the CPU and are not compute time). For a periodic task with instrumented computation time samples $c_1, c_2, \ldots, c_n$ across n observed cycles, the WCET estimate is:

$$C_i = \max_{1 \le k \le n} c_k \tag{1}$$

Period T_i was taken directly from the code where explicitly specified (`vTaskDelay`/`vTaskDelayUntil` arguments), and measured from consecutive log timestamps where period emerges from behavior rather than a fixed constant (`maintenance_task`, whose self-test hold was subsequently lengthened to a calibrated CPU-bound busy-loop, giving a measured period near 7030 ms). `blink_task` and `display_task` run at two distinct rates: a nominal 1000 ms period, and a 333 ms period during a 15-cycle emergency-override window triggered by `stall_button_task`. Both rates are analyzed separately in Section 3, since RMS schedulability must be checked against the worst-case (highest) rate a task can run at.

Task	C_i (µs)	T_i (nominal)	$U_i = C_i/T_i$	Priority	Core
`button_task`	1650	50 ms	0.0330	5	unpinned
`blink_task`	4399	1000 ms	0.0044	5	1
`display_task`	16168	1000 ms	0.0162	5	unpinned
`maintenance_task`	2,913,597*	7030 ms	0.4145	2 (low)	0
`telemetry_task`	68585	500 ms	0.1372	4 (med)	0
`stall_button_task`	6828	50 ms	0.1366	6 (high)	0

**Flagged value:* `maintenance_task`'s self-test was deliberately lengthened to a calibrated CPU-bound busy-loop (from an earlier design that used a plain 1 s `vTaskDelay`, which cannot be preempted meaningfully since sleeping tasks do not contend for the CPU). The 2,913,597 µs figure was measured *with* priority inheritance enabled, i.e. protected from preemption during the hold. An otherwise identical run with priority inheritance disabled measured 3,052,451 µs for the same nominal hold — a ∼139 ms inflation directly attributable to `telemetry_task` repeatedly preempting the lock-holder once the inheritance boost was removed (see Section 5). `stall_button_task`'s 6828 µs similarly reflects a heavier-contention run, against a quieter-run baseline of ∼4137 µs.
Each nominal-rate U_i is computed as:

$$
\begin{aligned}
U_{\text{button}} &= \frac{1650}{50{,}000} = 0.0330 \\
U_{\text{blink}} &= \frac{4399}{1{,}000{,}000} = 0.0044 \\
U_{\text{display}} &= \frac{16168}{1{,}000{,}000} = 0.0162 \\
U_{\text{maint}} &= \frac{2{,}913{,}597}{7{,}030{,}000} = 0.4145 \\
U_{\text{telemetry}} &= \frac{68585}{500{,}000} = 0.1372 \\
U_{\text{stall}} &= \frac{6828}{50{,}000} = 0.1366
\end{aligned}
$$

3 Liu & Layland RMS Schedulability Bound

For n periodic tasks scheduled under Rate-Monotonic Scheduling (RMS), Liu and Layland's sufficient (not necessary) condition for schedulability is:

$$
U_{\text{total}} = \sum_{i=1}^{n} \frac{C_i}{T_i} \;\leq\; n\left(2^{1/n} - 1\right) \tag{2}
$$

Step 1 — compute the bound for $n = 6$

$$
\begin{aligned}
n\left(2^{1/n} - 1\right) &= 6\left(2^{1/6} - 1\right) \\
&= 6\,(1.1225 - 1) \\
&= 6(0.1225) \\
&= 0.7348 \quad (73.48\%)
\end{aligned}
$$

Step 2a — nominal operation

With `blink_task` and `display_task` at their 1000 ms nominal period:

$$\begin{aligned} U_{\text{total}} &= U_{\text{button}} + U_{\text{blink}} + U_{\text{display}} + U_{\text{maint}} + U_{\text{telemetry}} + U_{\text{stall}} \\ &= 0.0330 + 0.0044 + 0.0162 + 0.4145 + 0.1372 + 0.1366 \\ &= 0.7019 \quad (74.19\%) \end{aligned}$$

Step 2b — emergency-mode operation

`blink_task` and `display_task` run at 333 ms (three times the nominal rate) for up to 15 cycles following a stall-button press. Recomputed at $T = 333$ ms:

$$U_{\text{blink,e}} = \frac{4399}{333{,}000} = 0.0132$$
$$U_{\text{display,e}} = \frac{16168}{333{,}000} = 0.0486$$

$$\begin{aligned} U_{\text{total,e}} &= U_{\text{button}} + U_{\text{blink,e}} + U_{\text{display,e}} + U_{\text{maint}} + U_{\text{telemetry}} + U_{\text{stall}} \\ &= 0.0330 + 0.0132 + 0.0486 + 0.4145 + 0.1372 + 0.1366 \\ &= 0.7331 \quad (78.31\%) \end{aligned}$$

Step 3 — compare against the bound

$$0.7348 = n\left(2^{1/n} - 1\right) \; < \; U_{\text{total}} = 0.7419 \; < \; U_{\text{total,e}} = 0.7331$$

Conclusion: after deliberately lengthening `maintenance_task`'s hold to make contention effects observable, the task set now exceeds the Liu–Layland sufficient bound in both its nominal (74.19%) and emergency-mode (78.31%) operating states — a materially different result from earlier design iterations, where utilization sat at 27–64% with comfortable margin. Two points are worth stating precisely rather than treating this as an outright failure:

- The bound is *sufficient, not necessary*: exceeding it does not prove the task set is unschedulable, only that this test can no longer guarantee it. A full Response Time Analysis (RTA), iteratively computing each task's worst-case response time against higher-priority interference, would be required to determine actual schedulability with certainty.
- The bound's validity assumes blocking time from lower-priority resource-holders is itself bounded — exactly the property priority inheritance provides. Section 5 measures the ∼139 ms cost of that assumption being violated when inheritance is disabled, which is the practical justification for the technique in certifiable real-time systems.

Note on an earlier design iteration

An earlier version of `telemetry_task` used an unbounded busy-loop whose measured execution time equaled its own period, producing $U_{\text{telemetry}} \approx 1.00$ and pushing U_{total} to approximately 113%—exceeding even the theoretical 100% CPU ceiling, not merely the 73.5% sufficient bound.

This was corrected by giving `telemetry_task` a fixed period via `vTaskDelayUntil` and reducing its busy-loop iteration count, bringing its WCET down to a small, bounded fraction of its own period. The RMS bound correctly identified a real design flaw prior to this fix; the current borderline result above stems from a separate, later change (lengthening `maintenance_task`'s hold), not a regression of this earlier issue.

4 Priority Inheritance: Measured Comparison

With `maintenance_task`, `telemetry_task`, and `stall_button_task` pinned to the same core, two otherwise identical runs were captured differing only in `USE_PRIORITY_INHERITANCE`:

	Inheritance ON (mutex)	Inheritance OFF (binary semaph
`[TELEMETRY]` lines during a contested wait	0 (both windows observed)	5 and 2 (two windows observe
Wait bounded by holder's remaining work only	Yes	No — stretched by each interrup

With inheritance enabled, `maintenance_task` is boosted to the waiting task's priority for the duration of the wait, making it un-preemptable by the medium-priority `telemetry_task` sharing its core — confirmed by the complete absence of interleaved `[TELEMETRY]` log lines during either contested window observed. With inheritance disabled, `telemetry_task` preempted the lock-holder repeatedly and unpredictably, and the wait grew by accumulation with no bound derivable from the code — the measured mechanism behind the $\sim$139 ms inflation noted in Section 2.